



Git & GitHub 完全入門ガイド

初心者のためのバージョン管理入門 ～ 基礎から実践まで

目次

- 
- 第1章: バージョン管理とは？
 - 第2章: Gitの基本概念
 - 第3章: Gitの環境構築
 - 第4章: Gitの基本操作
 - 第5章: ブランチの使い方
 - 第6章: GitHubとは？
 - 第7章: GitHubの基本操作
 - 第8章: プルリクエスト
 - 第9章: チーム開発の実践
 - 第10章: まとめ

第1章: バージョン管理とは？

なぜバージョン管理が必要なのか、その重要性を学びましょう



バージョン管理がない世界



ファイル名が「最終版」「最終版_改」
「最終版_本当の最終」に...



誰が何を変更したかわからない



以前のバージョンに戻せない



複数人での作業が大混乱

バージョン管理とは

ファイルの変更履歴を記録・管理するシステム



過去の任意の時点に戻れる



いつ、誰が、何を
変更したかを記録



複数人での同時作業が可能



変更の衝突を
検出・解決できる

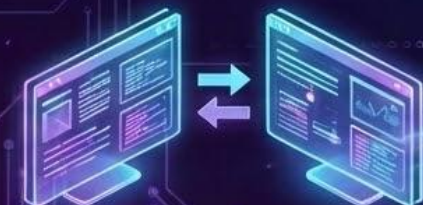
バージョン管理の例え：タイムマシン



好きな時点にいつでも戻れる



作業の「スナップショット」を保存



過去と現在を比較できる



間違いを簡単に取り消せる

まるでファイルのタイムマシン！

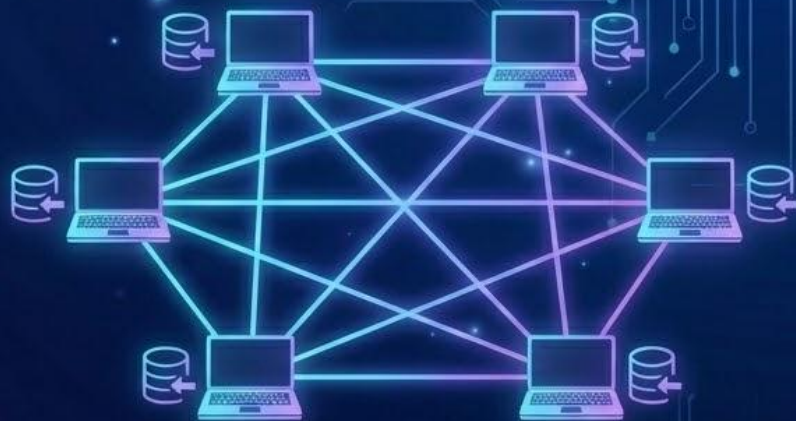
バージョン管理システムの種類

【集中型】



- SVN (Subversion)
- 1つのサーバーで全履歴を管理

【分散型】



- Git ← 現在最も人気！
- Mercurial
- 各自がフルの履歴を持つ

なぜGitが選ばれるのか



高速：ほとんどの操作がローカルで完結



堅牢：データの整合性を自動チェック



分散型：オフラインでも作業可能



ブランチ：複数の作業を並行して進められる



無料&オープンソース

第1章のまとめ



バージョン管理 = ファイルの変更履歴を管理



過去に戻れる、複数人で作業できる



Git = 最も人気のある分散型バージョン管理システム



高速・堅牢・柔軟

次は、Gitの基本概念を学びましょう！ >>

第2章: Gitの基本概念

Gitの重要な用語と概念を理解しましょう

リポジトリ (Repository)

📁 リポジトリ = Gitが管理する「倉庫」

📁 プロジェクトのファイル一式

🕒 ・全ての変更履歴

⚙️ ・設定情報

略して「リポ」「レポ」と呼ぶことも



ローカルとリモート (Local and Remote)

 ローカルリポジトリ
= 自分のPC上にあるリポジトリ

 リモートリポジトリ
= サーバー上にあるリポジトリ
(GitHub, GitLabなど)



この2つを同期させながら開発を進める

コミット (Commit)

 コミット = 変更の「セーブポイント」



 ある時点でのファイルの状態を保存

 メッセージ付きで記録できる

 後からいつでも戻れる

ゲームのセーブと同じイメージ! 

コミットの流れ (Commit Flow)

 ファイルを編集する  ステージング (変更を準備)  コミット (変更を確定)



「編集 → 準備 → 確定」の3ステップ

Gitの3つのエリア

1 作業ディレクトリ

2 ステージングエリア 📁

3 リポジトリ 📷



実際にファイルを
編集する場所



コミットする変更を
準備する場所



コミットした変更が
保存される場所

ステージングのイメージ

郵便の例えで考えてみよう

✍️ 手紙を書く = ファイルを編集

📬 投函口に入れる = ステージング

✉️ 配達される = コミット



投函前なら、まだ変更できる！

ブランチ (Branch)

🌳 ブランチ = 作業の「分岐」



🔗 メインの開発ラインから分岐

🔒 独立して作業できる

🔗 後で合流（マージ）できる

新機能開発やバグ修正を安全に行える

マージ (Merge)

✂ マージ=ブランチを「合流」させる



✂ 別々に作業した内容を統合

✂ 変更が競合する場合は手動で解決

🛡 履歴を保ちながら統合できる

分岐した道を一つに戻すイメージ

第2章のまとめ



リポジトリ = Gitの管理単位



コミット = 変更の保存ポイント



ステージング = コミット前の準備段階



ブランチ = 並行して作業するための分岐



マージ = ブランチを統合する操作

次は、Gitの環境構築を行きましょう！ ➡➡

第3章：Gitの環境構築

Gitをインストールして使えるようにしましょう



Gitのインストール (Windows)



1. git-scm.com にアクセス



2. 「Download for Windows」 をクリック



3. ダウンロードしたインストーラーを実行



4. 基本的にはデフォルト設定でOK



5. インストール完了！



※ Git Bashも一緒にインストールされます ➤



Gitのインストール (Mac)



方法1: Xcode Command Line Tools

```
> xcode-select -install
```



方法2: Homebrew (推奨)

```
> brew install git
```



方法3: 公式インストーラー

git-scm.com からダウンロード



インストールの確認



ターミナル（コマンドプロンプト）で確認：
`$ git --version`

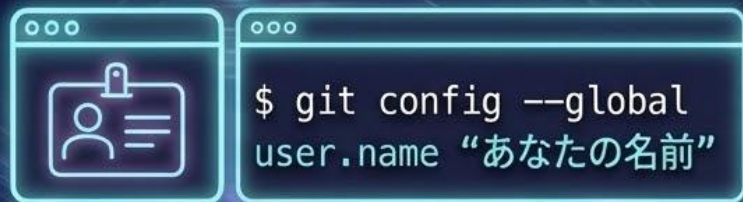
```
git version 2.43.0
```



↑バージョン番号が表示されればOK！

初期設定: ユーザー名とメール

コミット時に記録される情報を設定:

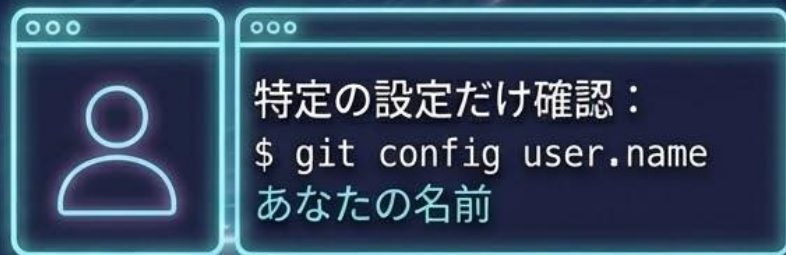


Commit History

この情報がコミット履歴に残ります

設定の確認

設定内容を確認：



設定が正しく反映されているか確認しましょう

第3章のまとめ



準備完了！次は実際にGitを使ってみましょう！🚀

第4章: Gitの基本操作

よく使うGitコマンドをマスターしましょう



git init - リポジトリの作成

新しいリポジトリを作成:

```
$ mkdir my-project  
$ cd my-project  
$ git init
```



my-project

「.git」フォルダが作成されれば成功！
(隠しフォルダなので見えないことも)

git status - 状態の確認

現在の状態を確認:

```
$ git status
```



迷ったら git status !

git add - ステージングする

変更をステージングエリアに追加:

```
$ git add ファイル名 # 特定のファイル  
$ git add .           # 全ての変更  
$ git add *.txt       # パターンで指定
```



コミットの「準備」をする操作

git commit - コミットする

ステージングした変更を確定：

```
$ git commit -m “コミットメッセージ”
```



Staged Files



Repository
(Commit History)

例：



```
$ git commit -m “ログイン機能を追加”
```



```
$ git commit -m “バグ修正: ボタンの表示崩れ”
```


良いコミットメッセージとは

コミット履歴を表示：

✗ 悪い例

`$ git log` # 詳細表示
`$ git log -oneline` # 1行表示
`$ git log -graph` # グラフ表示
`$ git log -n 5` # 最新5件

○ ———— ○ ———— ○ ? ○ ———— ○
qキーで終了

✓ 良い例

- “ユーザー登録フォームのバリデーション追加”
- “ヘッダーのレイアウト崩れを修正”
- “パスワードリセット機能を実装”

feature

git log - 履歴の確認

コミット履歴を表示：

```
$ git log
```

```
$ git log --oneline
```

```
$ git log --graph
```

```
$ git log -n 5
```

qキーで終了



詳細表示



1行表示



グラフ表示



最新5件

git diff - 差分の確認

変更内容を確認：

コマンド

```
$ git diff
```

```
$ git diff --staged
```

```
$ git diff HEAD
```

説明



ステージング前の変更



ステージング後の変更



全ての変更



- 削除された行



+ 追加された行

実践：初めてのコミット

コマンド

```
>_ $ mkdir my-first-repo
```

```
>_ $ cd my-first-repo
```

```
>_ $ git init
```

```
>_ $ echo "# My First Repo" > README.md
```

```
>_ $ git add README.md
```

```
>_ $ git commit -m "最初のコミット"
```



おめでとう！初コミット完了！

第4章のまとめ

基本コマンドまとめ：

-  \$ git init → リポジトリ作成
-  \$ git status → 状態確認
-  \$ git add → ステージング
-  \$ git commit → コミット
-  \$ git log → 履歴確認
-  \$ git diff → 差分確認



次はブランチを学びましょう！

なぜブランチを使うのか



本番コードを
壊さずに開発できる

→ 複数の機能を
← 同時に開発できる



失敗しても
簡単にやり直せる

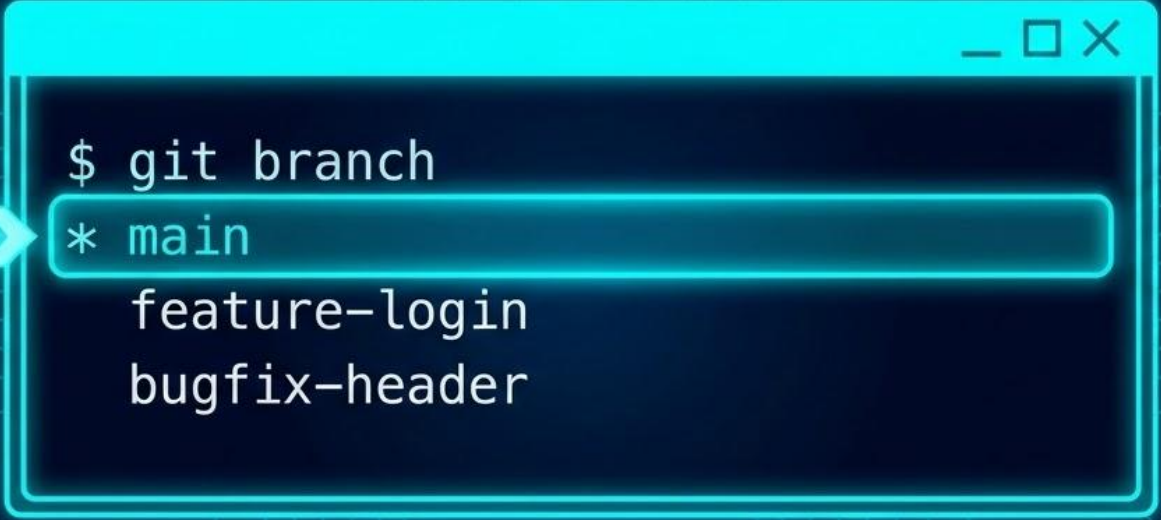


コードレビュー
がしやすくなる



チーム開発が
スムーズになる

git branch - ブランチの一覧



```
$ git branch
```

The image shows a terminal window with a blue title bar and standard window controls. The command `$ git branch` has been executed, and the output is displayed below it. A yellow arrow points from the text box below to the asterisk on the `* main` line.

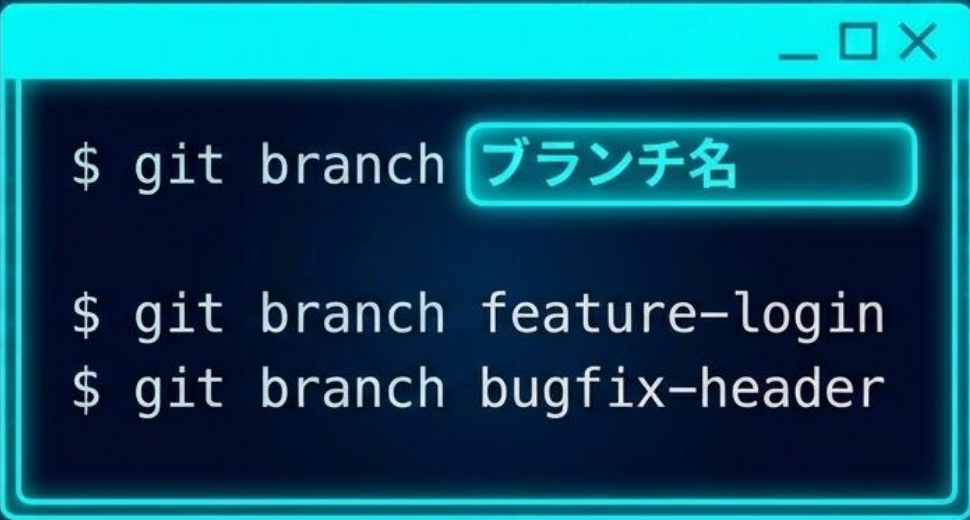
```
* main
```

```
feature-login
```

```
bugfix-header
```

「*」が付いているのが現在のブランチ

ブランチの作成

A terminal window with a blue title bar containing standard window controls. The text inside is white on a dark background. The first line shows a command with a placeholder box for the branch name. The next two lines show specific examples of branch names.

```
$ git branch ブランチ名
```

```
$ git branch feature-login
```

```
$ git branch bugfix-header
```



作成しただけでは移動しない

git checkout / git switch - ブランチの切り替え

```
$ git checkout ブランチ名 # 従来の方法
```

```
$ git switch ブランチ名 ✓ # 新しい方法 (推奨)
```

```
# 作成と同時に切り替え:
```

```
$ git checkout -b ブランチ名
```

```
$ git switch -c ブランチ名
```

main

feature-branch



git merge - ブランチの統合

```
$ git switch main          # mainに移動  
$ git merge feature-login # feature-loginを統合
```

feature-loginの変更がmainに取り込まれる

⌵ MERGED



コンフリクト（衝突）とは

同じ箇所を異なる方法で変更した場合に発生



Gitが「どちらを採用すべき？」と聞いている状態

コンフリクトの解決方法

```
1 class man {
2   <div>
3     <enfits="feature-login">
4
5     <<<<<<< HEAD
6     =====
7     >>>>>>> feature-login
8
9   </valuaes>
10 </div>
11 }
```

1. コンフリクトのある
ファイルを開く

```
1 class man {
2   <div>
3     <enfits="feature-login">
4
5     <<<<<<<
6     =====
7     >>>>>>>
8
9   </valuaes>
10 </div>
11 }
```



2. <<<<<<<, =====,
>>>>>>> を削除
3. 正しい内容に編集

```
$ git add .
```

4. git add で
ステージング

```
$ git commit -m  
"Merge conflict  
resolved"
```

5. git commit
でコミット

落ち着いて対処すれば大丈夫！ ✓

ブランチの削除

不要になったブランチを削除：

```
$ git branch -d ブランチ名
```

✔ # マージ済みのみ削除

```
$ git branch -D ブランチ名
```

⚠ # 強制削除



マージ後は整理しておくで見やすい 📁

[illegible][illegible][illegible]

\$ gi

\$ gi

[illegible][illegible]

```
#include <rot1.h>
int main() {
    char s[100];
    printf("Enter a string: ");
    fgets(s, sizeof s, stdin);
    printf("Rotated string: %s\n", rot1(s));
}
```


第6章: GitHubとは？

世界最大のソースコード管理プラットフォーム



Git と GitHub の違い



バージョン管理
システム (ツール)



ローカルで動作する
ソフトウェア



GitHub



Gitリポジトリの
ホスティングサービス

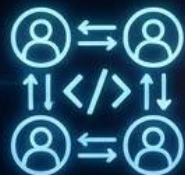


クラウド上でリポジ
トリを管理・共有

GitHubでできること



リポジトリのクラウド保存



チームでのコード共有



プルリクエストで
コードレビュー



イシュー（課題）の管理



プロジェクト管理



静的サイトの公開
(GitHub Pages)

第6章のまとめ



✓ **GitHub =**
Gitリポジトリの
ホスティング
サービス



✓ **Git $\neq$ GitHub**
(別物だが
一緒に使う)



✓ **コード共有、
チーム開発に
必須**



✓ **アカウント
作成は無料**



次はGitHubの基本操作を学びましょう！

第7章: GitHubの基本操作

ローカルとリモートを連携させましょう



リポジトリの作成 (GitHub上で)

1. 右上の「+」→「New repository」

2. リポジトリ名を入力 

3. Public  か Private  を選択

4. 「Create repository」をクリック



 READMEの追加はオプション

git clone - リポジトリをコピー

```
$ git clone  
https://github.com/ユーザー名/リポジトリ名.git
```



Remote
(GitHub)



Local
(ローカル)

👤 既存のプロジェクトに参加するときを使う

📋 「Code」 ボタンからURLをコピー



git remote - リモートの設定

```
$ git remote add origin https://github.com/user/repo.git
```



Local
Repository

git remote add



Remote
Repository
(origin)

確認：

```
$ git remote -v
```

i 「origin」はリモートの別名（慣習的に使用）

git push - リモートに送信



Local Repository
(ローカル)

```
$ git push origin main
```



Remote Repository
(GitHub)

🔗 初回は `-u` オプションで上流ブランチを設定：

```
$ git push -u origin main
```

✅ 以降はシンプルに：

```
$ git push だけでOK
```

git pull - リモートから取得



Local Repository
(ローカル)

```
$ git pull origin main
```



Remote Repository
(GitHub)

 他の人の変更を取得するときに使う：

最新の状態をチームと共有

 作業開始前に pull するのが良い習慣：

コンフリクトを未然に防ぐ

push と pull の関係



push : ローカルの変更を
リモートへ送信



pull : リモートの変更を
ローカルへ受信



双方向で同期を取りなが
ら開発を進める

第7章のまとめ



次はプルリクエストを学びましょう！

第8章：プルリクエスト

チーム開発の核となる機能を学びましょう



プルリクエスト (PR) とは



Feature Branch
(ブランチ)

「この変更をマージしてください」
というリクエスト

プルリクエスト (PR)



Main Branch
(メイン)



レビュー
(Review)

ブランチの変更を
メインに取り込む提案



承認
(Approval)

コードの品質を
保つ仕組み



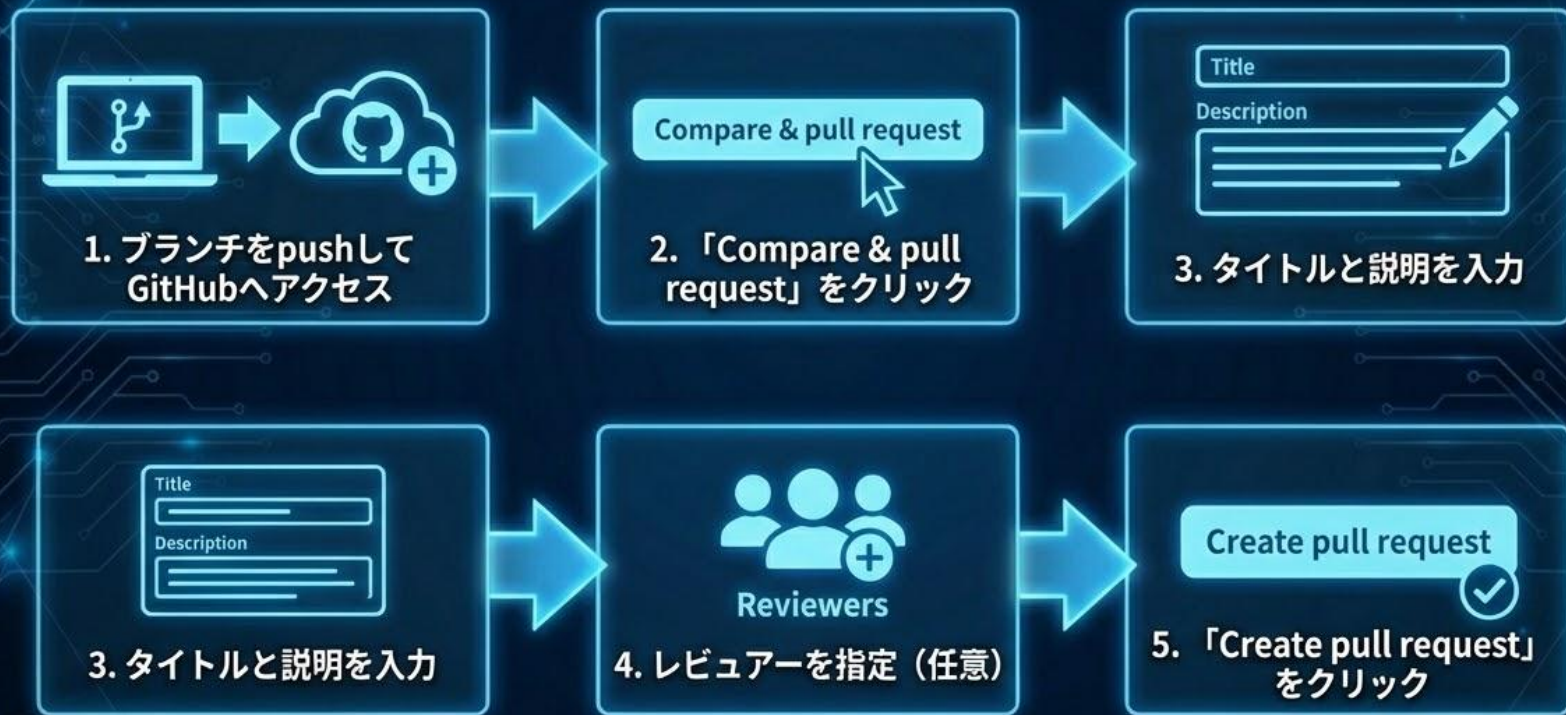
マージ
(Merge)

略して「PR」
「プルリク」と呼ぶ

PRの流れ



PRの作成方法



良いPRの書き方



1. 明確なタイトル
(何をしたか)



2. 変更の目的を説明



3. 変更内容の概要



4. テスト方法 (あれば)



5. スクリーンショット
(UI変更時)



レビュアーが
理解しやすいように！

コードレビューとは



他の開発者がコードをチェックすること

メリット：



バグの早期発見



コード品質の向上



知識の共有



チーム全体の
スキルアップ

レビューコメントの付け方



PRのマージ



第8章のまとめ



✓ PR = 変更をマージ
するリクエスト



✓ コードレビューで
品質を保つ



✓ 良いPRは
説明が明確



次はチーム開発の実践を学びましょう! ➡



第9章: チーム開発の実践

実際のチーム開発で役立つ知識を学びましょう

Git Flow (開発フロー)

代表的なブランチ戦略

main - 本番リリース用

develop - 開発用のメインブランチ

release/*
- リリース準備用

hotfix/*
- 緊急バグ修正用

feature/* - 新機能開発用



GitHub Flow (シンプル版)

よりシンプルな開発フロー：

main

1. mainから新しいブランチを作成



2. 変更をコミット



3. PRを作成



4. レビュー&
ディスカッション



5. デployして
テスト



6. mainにマージ

小規模チームにおすすめ！

イシュー (Issue) の活用



.gitignore ファイル

🚫 Gitで追跡しないファイルを指定



機密情報やビルド成果物は追跡しない！🔒

README.md の重要性

📖 プロジェクトの説明書



最初に見られるファイル！

フォーク (Fork) とは

🍴 他人のリポジトリを自分のアカウントにコピー



第9章のまとめ



次は全体のまとめです！

第10章: まとめ

Git & GitHubの学習を振り返りましょう



学んだこと (1/3)

基礎知識



学んだこと (2/3)

Gitの基本操作



基本コマンド



git log, diff
履歴と差分の確認



git branch, switch, merge
ブランチ操作と統合



コンフリクトの解決方法
衝突の解消

学んだこと (3/3)

🌐 GitHubとチーム開発



リモート操作
(clone, push, pull)



PR&コードレビュー



開発フロー
(Git Flow / GitHub Flow)



プロジェクト管理
(Issue, .gitignore, README)

よく使うコマンドまとめ (1/2)

【ローカル操作】



git init
リポジトリ作成



git status
状態確認



git add
ステージング



git commit
コミット



git log
履歴表示



git diff
差分表示

よく使うコマンドまとめ (2/2)

【ブランチ・リモート操作】



困ったときは



SOS git status を実行！

現在の状態を確認。



📖 git help コマンド名

ヘルプを表示。



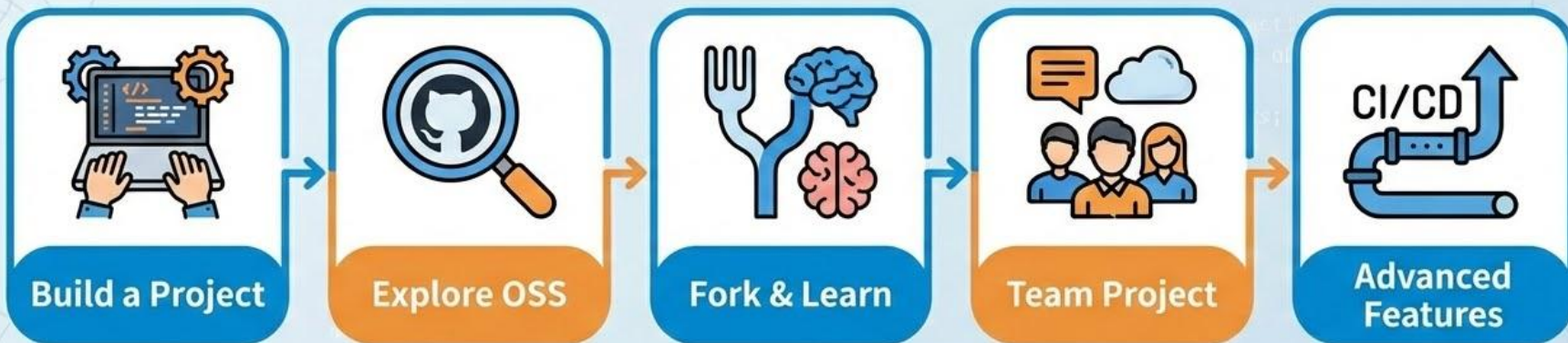
**🔍 エラーメッセージを
Google検索**

大抵の解決策が見つかる。



落ち着いて対処すれば大丈夫！

次のステップ



1. 実際にプロジェクト
を作ってみる

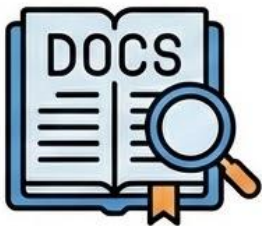
2. GitHubで
オープンソースを探索

3. 他の人のコードを
Forkして学ぶ

4. チームで
プロジェクトを始める

5. CI/CDなど
高度な機能を学ぶ

おすすめ学習リソース



公式ドキュメント

- git-scm.com/doc
- docs.github.com



インタラクティブ学習

- learngitbranching.js.org
- [GitHub Learning Lab](https://github.com/GitHubLearningLab)



書籍・チュートリアル

- Pro Git (無料オンライン書籍)

ありがとうございました！

🎉 **Git & GitHub 完全入門ガイド 修了！**

これであなたもGit & GitHubの
基礎をマスターしました。

実践あるのみ！

Happy Coding! 🚀



付録A: 便利なGitコマンド

知っておくと便利な追加コマンド集



```
(function f {  
  codestyle: "elegant"  
})  
state {  
  readLine = "read/cun, put"
```

```
})
```

```
citrus: {  
  codestyle: "elegant"
```


付録A: 便利なGitコマンド

git stash - 変更を一時退避

作業中の変更を一時的に保存：

```
$ git stash      # 退避  
$ git stash pop  # 復元  
$ git stash list # 一覧表示
```

💡 ブランチ切り替え前に便利！



git reset - コミットの取り消し

コミットを取り消す：

```
$ git reset --soft HEAD~1  
# コミットのみ取り消し  
$ git reset --mixed HEAD~1  
# ステージングも取り消し  
$ git reset --hard HEAD~1  
# 全て取り消し (注意！)
```

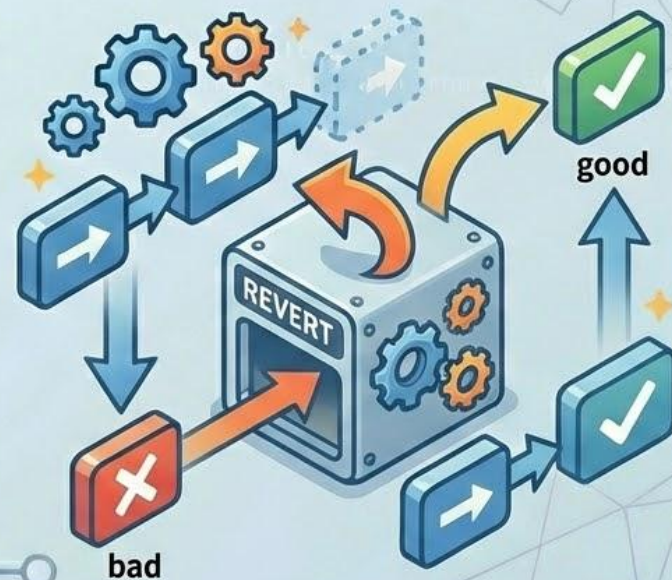
⚠ pushした後のresetは要注意



安全にコミットを取り消す：

```
$ git revert コミットID
```

- 指定したコミットを打ち消す新しいコミットを作成
- 履歴を消さないので安全
- push済みの変更を取り消すときに使う

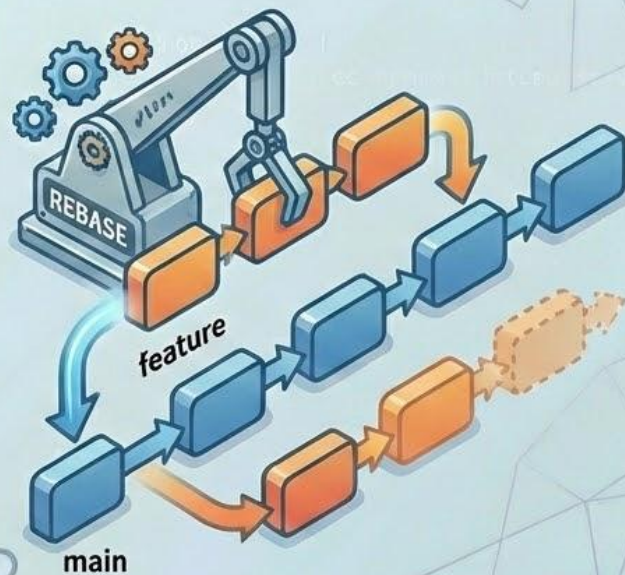


git rebase - コミット履歴を整理

ブランチの基点を移動：

```
$ git rebase main
```

- コミット履歴をきれいに保てる
- マージコミットが作られない
- ⚠ 共有ブランチでは使用注意
- 中級者向けコマンド

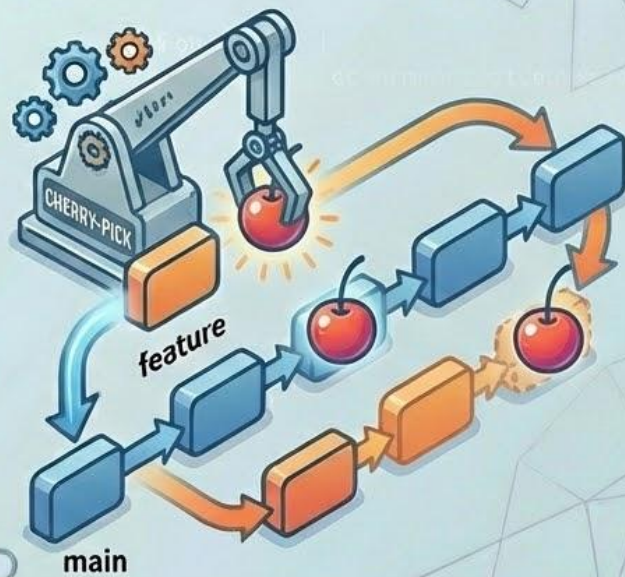


git cherry-pick - 特定のコミットを取り込む

特定のコミットだけを取り込む：

```
$ git cherry-pick コミットID
```

- 他のブランチの特定のコミットを取り込める
- バグ修正だけを別ブランチに適用したいときに便利
- 🍒 さくらんぼを摘むイメージ

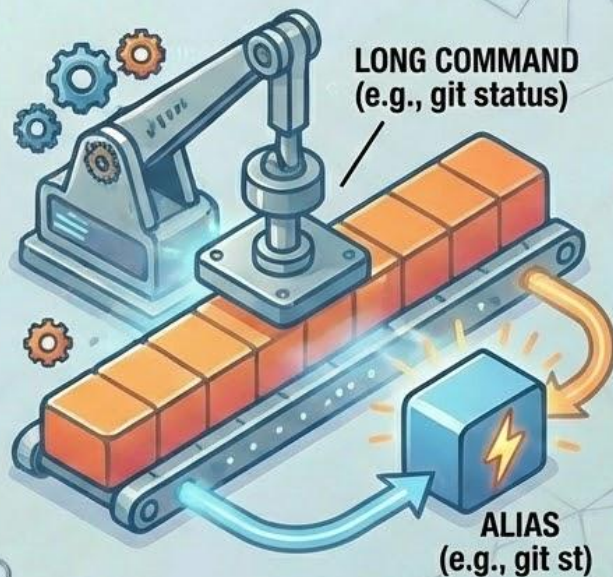


付録B: Gitのエイリアス設定

よく使うコマンドに短い名前を付ける：

```
$ git config --global alias.st status  
$ git config --global alias.co checkout  
$ git config --global alias.br branch  
$ git config --global alias.cm "commit -m"
```

これで **git st** で **git status** が実行できる！



付録C: GUIツールの紹介

コマンドが苦手な人向けのGUIツール：



 **GitHub Desktop**
(公式・無料)



 **Sourcetree**
(無料)



 **GitKraken**
(有料/無料版あり)



 **VS Code の Git 機能**
(内蔵Git機能)

視覚的にGitを操作できる！



付録D: SSH認証の設定

HTTPSの代わりにSSHを使う：



1. SSHキーを生成

```
$ ssh-keygen -t ed25519
```



2. 公開鍵をGitHubに登録
Settings → SSH and GPG keys



1. SSH キーを生成



3. SSH URLでclone

```
$ git clone  
git@github.com:user/repo.git
```

パスワード入力が必要になる！

付録E: よくあるエラーと解決策(1/2)

✗ “fatal: not a git repository”



```
$ git init
```

していない or 間違ったディレクトリ

✗ “error: failed to in you app”



```
$ git pull
```

先に git pull してからpush

✗ “error: failed to push some refs”



```
$ git pull
```

先に git pull してからpush

✗ “Please tell me who you are”



```
$ git config --global user.name "..."  
$ git config --global user.email "..."
```

ユーザー名とメールを設定する

エラーも怖くない！落ち着いて対処しよう。

付録E: よくあるエラーと解決策(2/2)

✖ “Your branch is behind”



```
$ git pull
```

で最新を取得

✖ “Connmction it”



```
$ git pull
```

で最新を取得

✖ “CONFLICT”



コンフリクトを手動で解決

✖ “Permission denied”



SSH設定やアクセス権限を確認

エラーは成長のチャンス！焦らず、一つずつ解決しよう。

ご清聴ありがとうございました



🎓 Git & GitHub 完全入門ガイド

これで基礎は完璧！
あとは実践あるのみです。
質問があればお気軽にどうぞ！



Happy Coding! 🚀 ✨

著作権・参考情報



本スライドは教育目的で作成されました

参考資料：



Git公式ドキュメント (git-scm.com)



GitHub公式ドキュメント (docs.github.com)



Pro Git Book

Git & GitHub 完全入門ガイド 全100枚